

Roadmap — Hackathon Agents, Everywhere

PromecTeam · Agente de diagnóstico y matching de precisión

Repo: github.com/MatiCS97/Hackathon-OpenAI---PromecTeam

1. Ya hecho antes del evento (setup, no funcionalidad central)

- Next.js 16 + Tailwind + CopilotKit instalado, build probado sin errores.
- Runtime de CopilotKit conectado a Claude (claude-sonnet-5) — texto y visión probados.
- Voyage AI conectado para embeddings — probado, devuelve vectores de 1024 dims.
- Script generador de dataset sintético de profesionales (5.000–20.000 perfiles).
- Repo GitHub público creado (vacío, listo para el primer push).
- CLAUDE.md con el contrato de datos, stack y decisiones ya escrito en la raíz del repo.

2. Qué falta construir (en vivo, durante el evento)

- Paso 1 — Diagnóstico: texto/foto/voz → JSON estructurado, incluye cotizador de costo y horas estimadas.
- Paso 2 — Matching: embedding del diagnóstico (Voyage) vs. embeddings de perfiles, + filtros duros (ubicación/disponibilidad).
- Paso 2b — Fallback: si el score de match es bajo, usar la tool nativa web_search de Claude para estimar costo con datos reales de internet.
- Paso 3 — Explicabilidad: por qué se eligió cada profesional, citando datos concretos.
- Frontend: reemplazar el placeholder por input de foto/texto, botón de voz (Web Speech API del navegador), y useCopilotAction/useCopilotReadable para mostrar todo como Generative UI dentro del chat.
- Demo host app: mini marketplace genérico para grabar el video sin depender de ChambaYa.

3. División de roles (equipo de 2 — ajustar si son más)

Persona	Módulo	Detalle
Vos	Frontend + Diagnóstico	UI, foto/voz, Paso 1 con cotizador. Ya tenés la base armada (route.ts, provider).
Compañero	Matching + Explicabilidad	Paso 2-3, embeddings Voyage, fallback web_search. Módulo aislado, no pisa tus archivos.
Ambos	Integración + Demo	14:45–15:15: unir todo, probar end to end, grabar video, armar submission.

4. Git: cuándo se hace push

- 11:15–11:30 — cada uno clona el repo, crea su propia rama (ej. diagnostico / matching).

- 11:30 en adelante — commits chicos y frecuentes, push seguido apenas algo compile. No se guarda todo para el final.
- 13:00–13:15 — Checkpoint 1: primer merge real, se detectan roturas de contrato JSON entre los dos módulos mientras hay tiempo de arreglarlas.
- 13:15–14:45 — se sigue construyendo y resolviendo lo que falló en el checkpoint.
- 14:45–15:15 — integración final en la demo host app + pulido.
- Nadie edita el mismo archivo al mismo tiempo sin avisar antes por chat del equipo.

5. Timeline del día

Hora	Actividad
10:00–10:30	Check-in, comida
10:30–11:00	Charla de apertura + briefing
11:00–11:15	Cerrar contrato de datos en equipo
11:15–11:30	Setup: repo, ramas, cada uno con su Claude Code
11:30–13:00	Build en paralelo por módulo
13:00–13:15	Checkpoint 1: integrar y detectar roturas
13:15–14:45	Seguir build + resolver lo que falló
14:45–15:15	Integración final + pulido
15:15–15:30	Grabar video de demo (2 min)
15:30–16:00	Submission: título, descripción, repo, video, post social

6. Claves compartidas (no subir a git)

- ANTHROPIC_API_KEY y VOYAGE_API_KEY: una sola de cada una para todo el equipo, no una por persona.
- Van en .env.local — ya está en .gitignore, nunca se commitea. Se comparten por chat privado del equipo.
- Antes del primer 'git add', correr 'git status' y confirmar que .env.local NO aparece en la lista.

7. Checklist final de submission

- Título del proyecto
- Descripción escrita clara
- Repo de GitHub público (ya creado)
- Video de demo de 2 minutos
- Post en redes sociales etiquetando a los partners del evento
- Poder explicar qué partes se construyeron durante el evento